

Contagem de Primos com OpenMP

Paralelismo de Laços e Observação de Race Conditions

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

August 25, 2026

1 Enunciado

Implementar um programa em C que conte quantos números primos existem entre 2 e um valor máximo N . Paralelizar o laço principal com `#pragma omp parallel for` sem alterar a lógica original. Comparar o tempo de execução e os resultados das versões sequencial e paralela. Observar diferenças no resultado e no desempenho. Refletir sobre correção e distribuição de carga.

2 Fundamentação teórica

2.1 Paralelismo de laços

O OpenMP distribui iterações de um laço `for` entre threads. A diretiva `#pragma omp parallel for` cria as threads e divide os índices. O escalonamento padrão (`static`) atribui blocos contíguos de tamanho igual a cada thread.

2.2 Race condition

Uma race condition ocorre quando threads escrevem na mesma variável sem sincronização. A CPU executa `total_primos_paralelo++` em três passos: LOAD (lê da memória), ADD (soma 1 no registrador) e STORE (grava na memória). Se duas threads executam o LOAD ao mesmo tempo, ambas leem o mesmo valor. Ambas somam 1 e gravam o mesmo resultado. O programa perde um incremento.

3 Experimento

O código usa $N = 5.000.000$. A função `eh_primo` verifica divisores ímpares até $\sqrt{n}$.

O programa executa duas versões:

1. **Sequencial:** Percorre o laço em uma thread.
2. **Paralela:** Adiciona `#pragma omp parallel for` ao mesmo laço, sem alterar a lógica.

O script varia o número de threads de 1 até o máximo disponível na máquina.

4 Código-fonte

```

1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <omp.h>
7
8 #define LIMITE_MAXIMO_PADRAO 5000000L
9
10 static double obter_tempo_segundos(void) {
11     struct timespec tempo_atual;
12     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
13     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
14 }
15
16 int eh_primo(long numero) {
17     if (numero <= 1) return 0;
18     if (numero == 2) return 1;
19     if (numero % 2 == 0) return 0;
20     for (long divisor = 3; divisor * divisor <= numero; divisor += 2) {
21         if (numero % divisor == 0) return 0;
22     }
23     return 1;
24 }
25
26 int main(int argc, char **argv) {
27     long limite_superior = argc > 1 ? atol(argv[1]) : LIMITE_MAXIMO_PADRAO
28     ;
29     if (limite_superior <= 0) {
30         fprintf(stderr, "Uso: %s [limite_superior]\n", argv[0]);
31         return 1;
32     }
33
34     int total_threads = omp_get_max_threads();
35
36     // VERSAO SEQUENCIAL
37     double tempo_inicio_sequencial = obter_tempo_segundos();
38     long total_primos_sequencial = 0;
39
40     for (long numero_atual = 2; numero_atual <= limite_superior; ++
41         numero_atual) {
42         if (eh_primo(numero_atual)) {
43             total_primos_sequencial++;
44         }
45     }
46
47     double tempo_execucao_sequencial = obter_tempo_segundos() -
48         tempo_inicio_sequencial;

```

```

46 // VERSAO PARALELA (sem alterar a logica original)
47 double tempo_inicio_paralelo = obter_tempo_segundos();
48 long total_primos_paralelo = 0;
49
50 #pragma omp parallel for
51 for (long numero_atual = 2; numero_atual <= limite_superior; ++
    numero_atual) {
52     if (eh_primo(numero_atual)) {
53         total_primos_paralelo++;
54     }
55 }
56 double tempo_execucao_paralelo = obter_tempo_segundos() -
    tempo_inicio_paralelo;
57
58 printf("N=%ld threads=%d seq_time=%.9f par_time=%.9f seq_count=%ld
    par_count=%ld\n",
59     limite_superior, total_threads,
60     tempo_execucao_sequencial, tempo_execucao_paralelo,
61     total_primos_sequencial, total_primos_paralelo);
62
63 return 0;
64 }

```

Listing 1: tarefa05.c

5 Resultados

5.1 Contagem

A versão sequencial contou 348.513 primos. A versão paralela retornou valores diferentes a cada execução. Com 2 threads, reportou cerca de 165.000. Com 8 threads, caiu para cerca de 40.000. Os incrementos se sobrescreveram na memória.

5.2 Tempo de execução

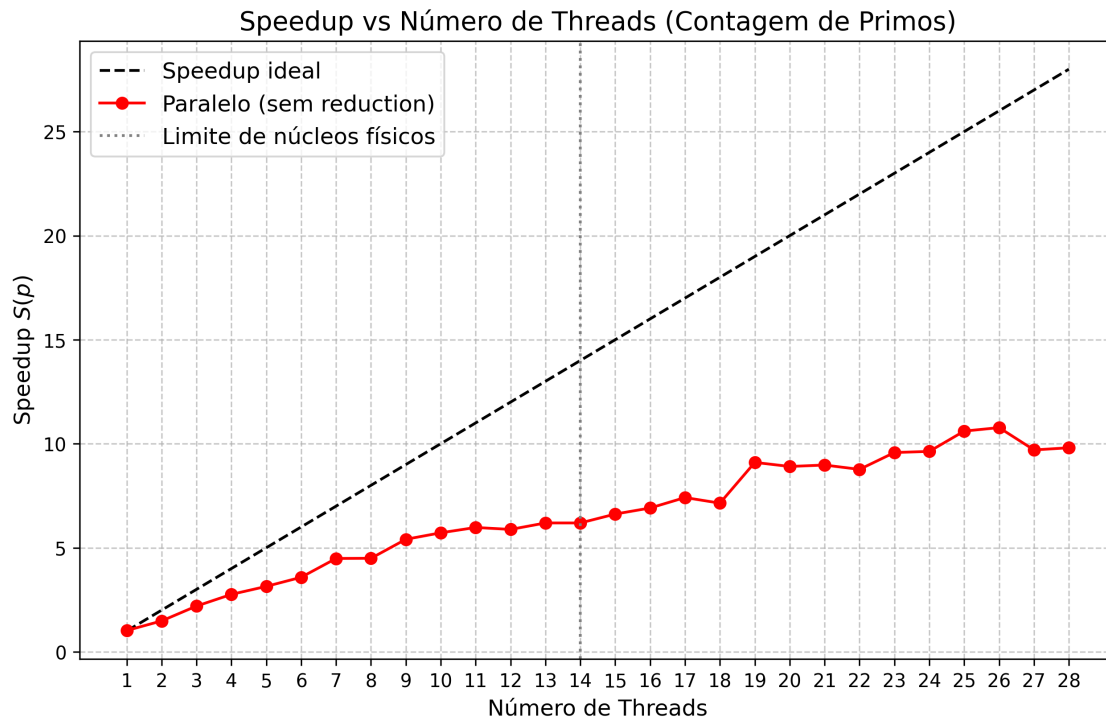


Figure 1: Speedup da versão paralela. A curva fica abaixo do ideal. O resultado está incorreto devido à race condition.

6 Discussão

6.1 A race condition é silenciosa

O programa compila e executa sem erros visíveis. Ele imprime um número no terminal. O número está errado, mas o programa não avisa. O programador detecta o problema comparando com a versão sequencial.

6.2 Desbalanceamento de carga

A Figura 1 mostra o speedup abaixo da reta ideal. O escalonamento estático divide os índices em blocos contíguos. A thread que recebe os números maiores gasta mais ciclos na ALU. As threads com números menores terminam antes e ficam ociosas.

6.3 Correção

A cláusula `reduction(+:total_primos_paralelo)` resolve a race condition. O compilador cria uma cópia privada da variável para cada thread. No fim do laço, o OpenMP soma os subtotais. O escalonamento dinâmico (`schedule(dynamic)`) equilibra a carga.

7 Conclusão

O `#pragma omp parallel for` distribui as iterações entre threads sem alterar a lógica do laço. O resultado ficou incorreto porque as threads escrevem na mesma variável sem sincronização. A race condition não gera erro visível. A correção exige mecanismos como `reduction`. O desbalanceamento de carga reduz o speedup abaixo do ideal.